CHAPTER THREE

SYSTEM ANALYSIS, DESIGN AND METHODOLOGY

3.0 Overview

This chapter presents the methodology adopted for the development of the Community Incident Reporting System (CIRS). It discusses the software development methodology employed, functional and non-functional requirements, system modelling tools, use case diagrams, flowcharts, and the architectural design of the proposed system. The chapter provides a blueprint for the implementation of the system and demonstrates how the research objectives were achieved.

3.1 Software Development Methodology

The Agile Software Development Methodology was adopted for the development of the Community Incident Reporting System. Agile is an iterative and incremental software development approach that emphasizes flexibility, collaboration, continuous feedback, and rapid delivery of functional software (Pressman & Maxim, 2020).

The methodology was selected because the proposed system required continuous refinement of requirements, user interface improvements, integration of geospatial technologies, and frequent testing throughout the development lifecycle. Unlike the Waterfall model, Agile allows developers to make adjustments during development without restarting the entire project.

Reasons for Choosing Agile Methodology

The Agile methodology was selected for the following reasons:

1. **Flexibility:** Requirements could be modified as new insights emerged during development.
2. **Incremental Development:** The system was developed in modules such as user management, incident reporting, mapping, and administration.
3. **Continuous Testing:** Each module was tested before integrating it into the complete system.
4. **User Feedback:** Feedback from prospective users could be incorporated into subsequent development iterations.

5. **Faster Delivery:** Functional components were delivered progressively rather than waiting for the entire system to be completed.

Application of Agile Methodology in the Study

The development process was divided into several iterations:

- **Iteration 1: Requirements Gathering and Analysis**

  The first iteration focused on gathering and analyzing the requirements for the Community Incident Reporting System. Information was obtained from the problem statement, research objectives, and literature review to identify the challenges associated with existing incident reporting methods. During this phase, the system's stakeholders, including community members and administrators, were identified, and their needs were analyzed. The functional and non-functional requirements of the system were also documented to serve as a foundation for the subsequent stages of development.

- **Iteration 2: System Design and Database Design**

  The second iteration involved designing the overall structure of the system and its database. System models such as use case diagrams, activity diagrams, and architectural diagrams were developed to illustrate how the system would operate. Additionally, the database schema was designed to define how user information, incident reports, location data, and incident statuses would be stored and managed. This phase provided a clear blueprint for the implementation of the system.

- **Iteration 3: User Authentication and Account Management**

  The third iteration focused on developing the user authentication and account management module. Features such as user registration, login, password management, and profile updates were implemented to ensure secure access to the system. Authentication mechanisms were incorporated to verify user identities and restrict access to authorized users. This module established the security foundation required for managing user activities within the platform.

- **Iteration 4: Incident Reporting Module Development**

  During the fourth iteration, the core functionality of the system—the incident reporting module—was developed. This module enables users to submit incident reports by providing details such as incident type, description, location, and supporting images. Input validation mechanisms were implemented to ensure the accuracy and completeness of submitted information. The module was designed to capture incident data efficiently and store it in the database for further processing and management.

- **Iteration 5: GIS and Mapping Integration**

  The fifth iteration focused on integrating geospatial technologies into the system. Mapping services and GIS tools were incorporated to display reported incidents on an interactive map. GPS coordinates captured during incident submission were used to accurately pinpoint incident locations. Additional features such as map markers, incident clustering, and hotspot visualization were implemented to support spatial analysis and improve situational awareness for authorities and community members.

- **Iteration 6: Administrative Dashboard Development**

  The sixth iteration involved the development of the administrative dashboard used by authorized personnel to manage incident reports. The dashboard provided functionalities for viewing, verifying, updating, and resolving reported incidents. It also included reporting and analytics features that allow administrators to monitor incident trends and assess community safety conditions. This module serves as the central management interface for effective incident administration and decision-making.

- **Iteration 7: Testing, Debugging, and Evaluation**

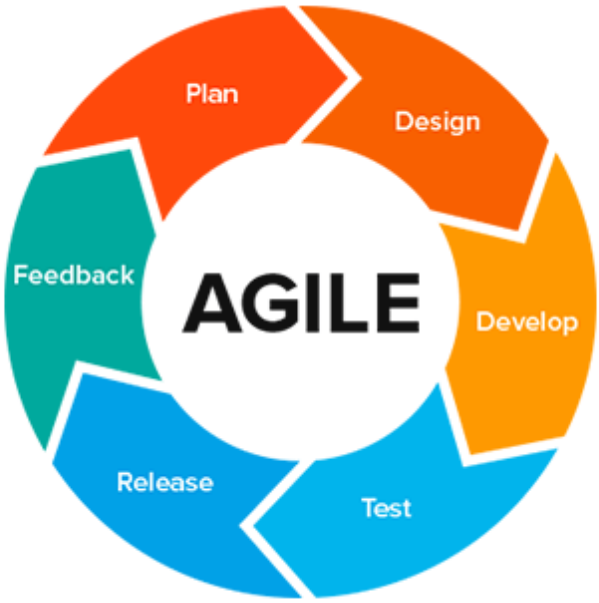
  The final iteration focused on testing, debugging, and evaluating the developed system. Functional testing was conducted to verify that all modules performed according to the specified requirements. Identified errors and system issues were corrected through debugging to improve reliability and performance. The system was then evaluated based

on usability, functionality, and user satisfaction to determine whether it successfully achieved the objectives of providing an efficient geospatially enabled community incident reporting platform.

The Agile methodology enabled the researcher to continuously evaluate progress and improve the system throughout development.

Figure 1

An illustration of the flow of Agile development methodology



3.2 Functional Requirements

Functional requirements describe the specific services and operations that the system must perform.

Citizen/User Functional Requirements

1. The system shall allow users to register accounts.
2. The system shall allow users to log into the platform.
3. The system shall allow users to report incidents.
4. The system shall capture GPS coordinates automatically during incident submission.

5. The system shall allow users to upload images relating to incidents.
6. The system shall allow users to view incidents on an interactive map.
7. The system shall allow users to track the status of reported incidents.
8. The system shall allow users to update their profile information.
9. The system shall notify users when incident statuses change.

Administrator Functional Requirements

1. The web application shall allow administrators to log into the management dashboard.
2. The web application shall allow administrators to view all submitted incidents.
3. The web application shall allow administrators to verify reported incidents.
4. The web application shall allow administrators to update incident statuses.
5. The web application shall allow administrators to categorize incidents.
6. The web application shall generate incident statistics and reports.
7. The web application shall display incident hotspots on a map.
8. The web application shall manage user accounts.
9. The web application shall search and filter incident records.

Maintenance Team Functional Requirements

1. The web application shall allow maintenance team lead to log into the maintenance dashboard.
2. The web application shall allow maintenance team to view all ssigned incidents.
3. The web application shall allow maintenance team to update incident statuses.
4. The web application shall generate incident statistics and reports.
5. The web application shall display incident hotspots on a map.
6. The web application shall search and filter incident records.

3.3 Non-Functional Requirements

3.3.1 Usability Requirements

1. The web application shall provide an intuitive and user-friendly interface.
2. The web application shall be easy to learn and use by community members and authorities.

3.3.2 Performance Requirements

1. The web application shall process incident submissions within five seconds.
2. The web application shall support multiple concurrent users without significant degradation in performance.

3.3.3 Reliability Requirements

1. The web application shall maintain data integrity during transactions.
2. The web application shall recover gracefully from unexpected failures.

3.3.4 Security Requirements

1. The web application shall require authentication before accessing protected resources.
2. The web application shall encrypt user passwords.
3. The web application shall restrict administrative functions to authorized users only.

3.3.5 Availability Requirements

1. The web application shall be accessible 24 hours a day through internet-enabled devices.
2. The web application shall provide regular backups of incident data.

3.3.6 Maintainability Requirements

1. The web application shall be modular and easy to maintain.
2. Future features should be integrated with minimal changes to existing modules.

3.4 Use Cases and Flowchart Diagrams

This section presents the visual and functional modelling of the Community Incident Reporting System using use case diagrams and flowchart diagrams. These diagrams are important in system

design as they provide a clear representation of how different users interact with the system and how processes are executed internally.

The **use case diagrams** illustrate the interactions between the system and its main actors, namely the User, Administrator, and Maintenance Team Lead. They show the different functions each actor can perform within the system such as user registration, incident reporting, incident verification, status updates, and report generation. This helps to clearly define the system's functional boundaries and ensures that all user requirements are properly captured and represented.

The **flowchart diagrams**, on the other hand, describe the step-by-step flow of processes within the system. They visually represent how an incident is reported, processed, verified, assigned, and resolved. Flowcharts help to break down complex processes into simpler steps, making it easier to understand system logic and workflow progression.

Together, the use case and flowchart diagrams provide a comprehensive understanding of the system's functional behaviour and process flow, serving as a bridge between system requirements and implementation.

3.4.1 Use Case Diagram

Figure 2

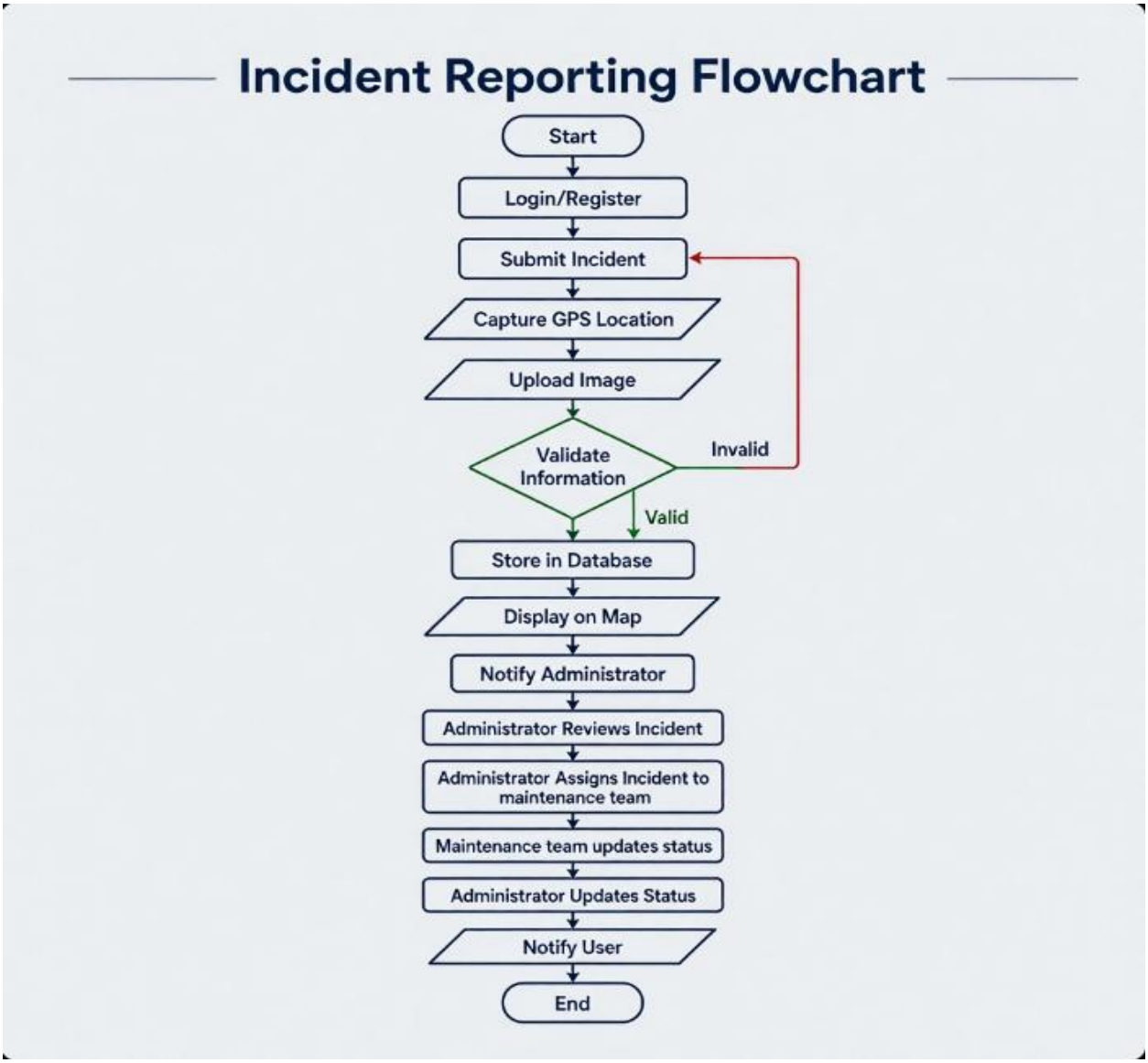
A Use Case diagram describing how users interact with the system



3.4.2 Incident Reporting Flowchart

Figure 3

A graphical representation of the step-by-step process users go through to report incidents and resolve them.



3.5 System Architectural Design

System architecture refers to the structural design of a software system, showing how different components interact to achieve the intended functionality. The proposed Community Incident Reporting System (CIRS) adopts a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer. This architectural approach was selected because it promotes

modularity, scalability, maintainability, and efficient separation of concerns. Each layer performs specific functions while communicating with the other layers to ensure seamless system operation. The architecture enables users to interact with the system through a web interface, while business logic and data processing are handled by the application layer. The data layer is responsible for storing and managing all information required by the system. This separation of responsibilities enhances system security, simplifies maintenance, and allows future upgrades without significantly affecting other components of the system.

3.5.1 System Architecture

The Community Incident Reporting System utilizes a three-tier architecture composed of the Presentation Layer, Application Layer, and Data Layer. The interaction among these layers facilitates efficient communication between users, system services, and data storage mechanisms.

- **Presentation Layer**

  The Presentation Layer serves as the front-end component of the system and provides the interface through which users interact with the application. This layer is responsible for displaying information, receiving user inputs, and communicating requests to the application layer for processing. It focuses on providing a user-friendly and responsive experience for both community members and system administrators.

  The presentation layer consists of a web-based interface developed using modern web technologies, allowing users to access the system through internet-enabled devices such as computers, tablets, and smartphones. Through this interface, users can register accounts, log in, submit incident reports, upload supporting images, and monitor the status of previously reported incidents.

  Additionally, the presentation layer includes an administrative dashboard that enables authorized personnel to manage incident reports, verify submitted information, update incident statuses, and generate reports. The dashboard provides administrators with a centralized platform for monitoring community incidents and making informed decisions.

A key component of this layer is the interactive GIS map interface. The map allows users and administrators to visualize incident locations geographically. Incident markers are displayed on the map based on GPS coordinates submitted during incident reporting. The mapping component enhances situational awareness by enabling users to identify incident locations, distribution patterns, and potential hotspots within the community.

- **Application Layer**

  The Application Layer acts as the core processing unit of the system and contains all business logic, rules, and services required for system operations. It serves as the intermediary between the presentation layer and the data layer by processing user requests, validating information, and coordinating system activities.

  One of the primary functions of this layer is user authentication and authorization. It verifies user credentials during login and ensures that only authorized individuals can access restricted functionalities within the system. This enhances system security and protects sensitive information from unauthorized access.

  The application layer also manages incident-related operations. When users submit incident reports, the layer validates the entered information, processes uploaded files, captures location data, and stores the information in the database. It further supports incident verification and status management by allowing administrators to review reports and update their progress through various stages such as Pending, Verified, In Progress, and Resolved.

  Another important responsibility of the application layer is report generation and analytics. It processes stored incident data and generates summaries, statistics, and visual reports that assist authorities in understanding incident trends and making evidence-based decisions.

  The application layer additionally handles GIS processing activities. It interacts with mapping services and geospatial APIs to convert coordinates into map locations, display incident markers, perform clustering, and identify areas with high concentrations of

incidents. This functionality supports spatial analysis and improves emergency response planning.

Notification management is also performed within this layer. The system automatically sends alerts and updates to users whenever significant changes occur, such as the verification or resolution of a reported incident. This feature improves transparency and keeps users informed about the status of their submissions.

- **Data Layer**

  The Data Layer is responsible for storing, retrieving, updating, and managing all information used by the Community Incident Reporting System. It serves as the foundation upon which the entire system operates by ensuring that data is organized, secure, and readily accessible when required.

  The layer stores user information, including registration details, login credentials, profile data, and user roles. This information is used for authentication and access control throughout the system.

  Incident records are also maintained within the data layer. Each record contains details such as the incident category, description, date and time of occurrence, current status, and information about the reporting user. These records form the primary dataset used for incident management and reporting activities.

  Since the system incorporates geospatial technology, the data layer stores geographic coordinates captured during incident submission. These coordinates enable accurate mapping and spatial analysis of incidents. The database supports the storage and retrieval of location-based information required by the GIS component of the system.

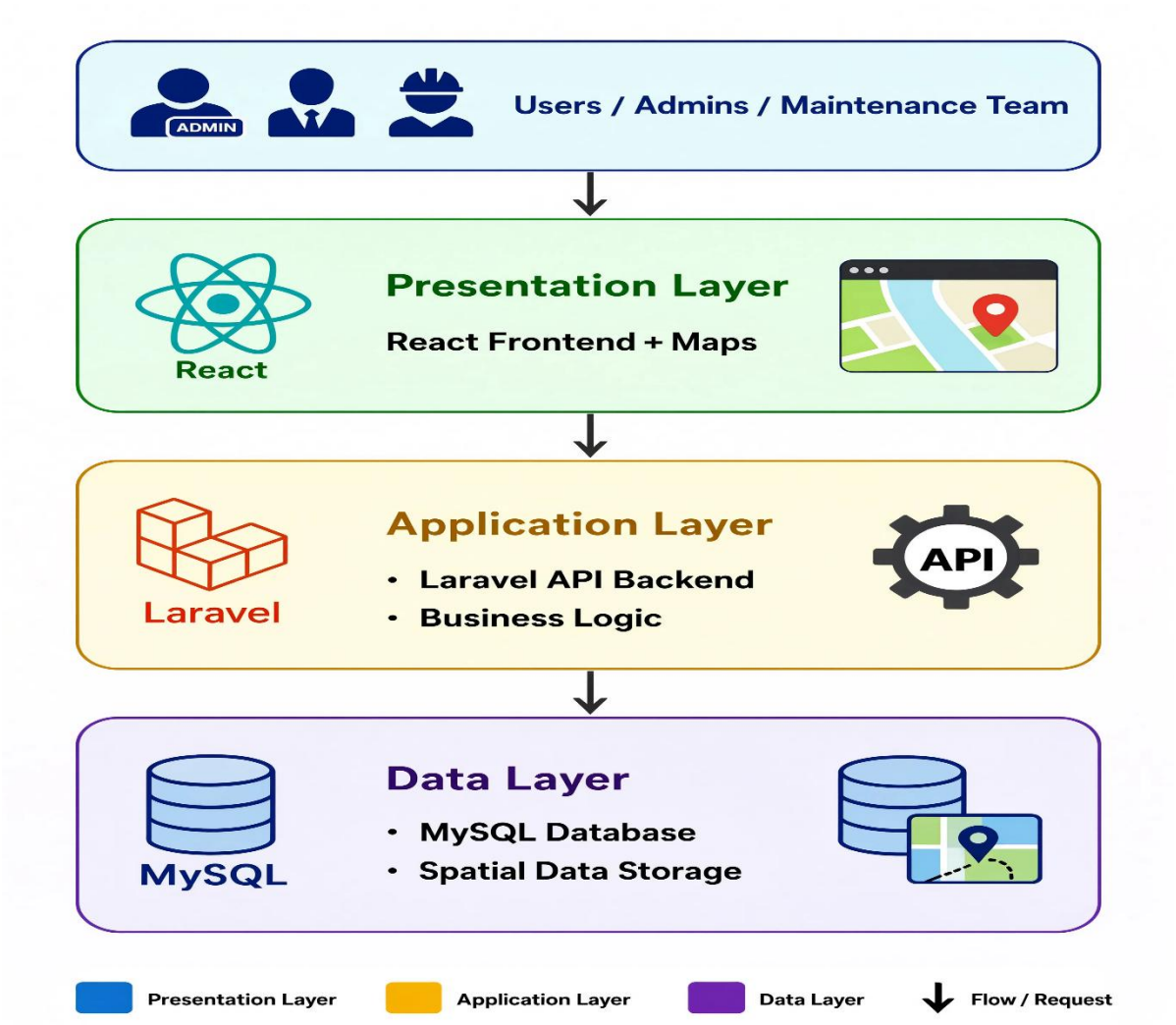
  Supporting media files such as photographs uploaded by users are also managed within this layer. These images provide additional evidence and context for reported incidents, enabling authorities to better assess situations before taking action.

Furthermore, the system maintains an incident status history that records all updates made to a report throughout its lifecycle. This historical information improves accountability, transparency, and auditability by allowing administrators and users to track actions taken on each incident from submission to resolution.

Through efficient data management and storage mechanisms, the data layer ensures the integrity, consistency, and availability of information required for the successful operation of the Community Incident Reporting System.

Figure 4

A diagrammatic representation of the system's architecture



References

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.

Dennis, A., Wixom, B. H., & Tegarden, D. (2020). *Systems analysis and design: An object-oriented approach with UML* (6th ed.). Wiley.

Rosenblatt, H. J. (2017). *Systems analysis and design* (12th ed.). Cengage Learning.